

EXPERIMENT 1: Write the python program to solve 8-Puzzle problem

Program

```
def print_state(state):
    for i in range(0, 9, 3):
        print(state[i:i+3])
    print() # blank line

def get_neighbors(state):
    neighbors = []
    blank_idx = state.index(0) # position of the empty tile
    row, col = divmod(blank_idx, 3)

    # possible moves: up, down, left, right
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dr, dc in moves:
        new_r, new_c = row + dr, col + dc
        if 0 <= new_r < 3 and 0 <= new_c < 3:
            new_blank = new_r * 3 + new_c
            new_state = list(state)
            # swap blank with the tile
            new_state[blank_idx], new_state[new_blank] = new_state[new_blank],
            new_state[blank_idx]
            neighbors.append(tuple(new_state))
```

```
return neighbors
```

```
def bfs(start, goal):
```

```
    from collections import deque
```

```
    queue = deque()
```

```
    queue.append(start)
```

```
    visited = {start: None}
```

```
    while queue:
```

```
        current = queue.popleft()
```

```
        if current == goal:
```

```
            path = []
```

```
            while current is not None:
```

```
                path.append(current)
```

```
                current = visited[current]
```

```
            path.reverse()
```

```
            return path
```

```
    for neighbor in get_neighbors(current):
```

```
        if neighbor not in visited:
```

```
            visited[neighbor] = current
```

```
            queue.append(neighbor)
```

```
return None
```

```
def is_solvable(state):
```

```
    inv = 0
```

```
    flat = [x for x in state if x != 0]
```

```
    for i in range(len(flat)):
```

```
        for j in range(i + 1, len(flat)):
```

```
            if flat[i] > flat[j]:
```

```
                inv += 1
```

```
    return inv % 2 == 0
```

```
if __name__ == "__main__":
```

```
    start_state = (1, 2, 3,
```

```
                  4, 5, 6,
```

```
                  0, 7, 8)
```

```
    goal_state = (1, 2, 3,
```

```
                 4, 5, 6,
```

```
                 7, 8, 0)
```

```
    print("Start state:")
```

```
    print_state(start_state)
```

```
    if not is_solvable(start_state):
```

```
        print("This puzzle configuration is not solvable.")
```

```
    else:
```

```
        print("Solving...")
```

```
solution = bfs(start_state, goal_state)
```

```
if solution is None:
```

```
    print("No solution found.")
```

```
else:
```

```
    print(f"Solved in {len(solution)-1} moves:")
```

```
    for step, s in enumerate(solution):
```

```
        print(f"Step {step}:")
```

```
        print_state(s)
```

sample input:

Start state:

(1, 2, 3)

(4, 5, 6)

(0, 7, 8)

Output:

Solved in 2 moves:

Step 0:

(1, 2, 3)

(4, 5, 6)

(0, 7, 8)

Step 1:

(1, 2, 3)

(4, 5, 6)

$(7, 0, 8)$

Step 2:

$(1, 2, 3)$

$(4, 5, 6)$

$(7, 8, 0)$

EXPERIMENT 2: Write the python program to solve 8-Queen problem

Program

```
def print_board(board):
```

```
    """Print the chessboard with Q for queens and . for empty squares."""
```

```
    for row in board:
```

```
        line = " ".join("Q" if col == 1 else "." for col in row)
```

```
        print(line)
```

```
    print() # blank line after each board
```

```
def is_safe(board, row, col):
```

```
    """Check if placing a queen at (row, col) is safe."""
```

```
    # check this row on left side
```

```
    for c in range(col):
```

```
        if board[row][c] == 1:
```

```
            return False
```

```
    # check upper diagonal on left side
```

```
    r, c = row, col
```

```
    while r >= 0 and c >= 0:
```

```
        if board[r][c] == 1:
```

```
            return False
```

```
        r -= 1
```

```
        c -= 1
```

```
# check lower diagonal on left side
```

```
r, c = row, col
```

```
while r < 8 and c >= 0:
```

```
    if board[r][c] == 1:
```

```
        return False
```

```
    r += 1
```

```
    c -= 1
```

```
return True
```

```
def solve_queens(board, col):
```

```
    """Recursive back-tracking to place queens column by column."""
```

```
    # base case: all queens are placed
```

```
    if col >= 8:
```

```
        return True
```

```
    # try placing queen in each row of this column
```

```
    for r in range(8):
```

```
        if is_safe(board, r, col):
```

```
            board[r][col] = 1          # place queen
```

```
            if solve_queens(board, col + 1): # recur for next column
```

```
                return True
```

```
            board[r][col] = 0          # backtrack
```

```
return False # no placement worked in this column
```

```

def main():
    # create an empty 8x8 board (0 = empty, 1 = queen)
    board = [[0 for _ in range(8)] for _ in range(8)]

    if solve_queens(board, 0):
        print("One solution for the 8-Queens problem:")
        print_board(board)
    else:
        print("No solution found.")

if __name__ == "__main__":
    main()

```

Output:

One solution for the 8-Queens problem:

```

Q.....
.....Q.
....Q...
.....Q
.Q.....
...Q....
.....Q..
..Q.....

```


EXPERIMENT 3: Write the python program for Water Jug Problem

Program

```
from collections import deque
```

```
def pour(state, from_jug, to_jug, capacities):
```

```
    """
```

```
    Return the new state after pouring water from one jug to another.
```

```
    state: (x, y) – current amount in jug1 and jug2
```

```
    from_jug, to_jug: 0 for jug1, 1 for jug2
```

```
    capacities: (cap1, cap2)
```

```
    """
```

```
    x, y = state
```

```
    cap1, cap2 = capacities
```

```
    if from_jug == 0 and to_jug == 1:
```

```
        transfer = min(x, cap2 - y)
```

```
        return (x - transfer, y + transfer)
```

```
    elif from_jug == 1 and to_jug == 0:
```

```
        transfer = min(y, cap1 - x)
```

```
        return (x + transfer, y - transfer)
```

```
    else:
```

```
        return state
```

```
def get_neighbors(state, capacities):
```

```
    """Generate all possible next states from the current state."""
```

```
x, y = state
```

```
cap1, cap2 = capacities
```

```
neighbors = []
```

```
neighbors.append((cap1, y))
```

```
neighbors.append((x, cap2))
```

```
neighbors.append((0, y))
```

```
neighbors.append((x, 0))
```

```
neighbors.append(pour(state, 0, 1, capacities))
```

```
neighbors.append(pour(state, 1, 0, capacities))
```

```
return neighbors
```

```
def bfs(start, goal, capacities):
```

```
    """Breadth-first search to find a sequence of moves."""
```

```
    queue = deque()
```

```
    queue.append((start, []))
```

```
    visited = {start}
```

```
    while queue:
```

```
        (cur_state, path) = queue.popleft()
```

```
        if cur_state == goal:
```

```
            return path + [cur_state]
```

```

    for nxt in get_neighbors(cur_state, capacities):
        if nxt not in visited:
            visited.add(nxt)
            queue.append((nxt, path + [cur_state]))

    return None

def main():

    capacities = (4, 3)
    start_state = (0, 0)
    goal_state = (2, 0)

    print(f'Capacities: Jug1 = {capacities[0]} L, Jug2 = {capacities[1]} L')
    print(f'Start state: {start_state}')
    print(f'Goal state: {goal_state}\n')

    solution = bfs(start_state, goal_state, capacities)

    if solution is None:
        print("No solution found.")
    else:
        print(f'Solution found in {len(solution)-1} steps:')
        for i, state in enumerate(solution):
            print(f'Step {i}: Jug1 = {state[0]} L, Jug2 = {state[1]} L')

```

```
if __name__ == "__main__":  
    main()
```

sample input:

Capacities: Jug1 = 4 L, Jug2 = 3 L

Start state: (0, 0)

Goal state: (2, 0)

Output:

Solution found in 6 steps:

Step 0: Jug1 = 0 L, Jug2 = 0 L

Step 1: Jug1 = 0 L, Jug2 = 3 L

Step 2: Jug1 = 3 L, Jug2 = 0 L

Step 3: Jug1 = 3 L, Jug2 = 3 L

Step 4: Jug1 = 4 L, Jug2 = 2 L

Step 5: Jug1 = 0 L, Jug2 = 2 L

Step 6: Jug1 = 2 L, Jug2 = 0 L

EXPERIMENT 4 Write the python program for Cript-Arithmetic problem

Program

```
import itertools
```

```
def word_to_number(word, mapping):
```

```
    """Convert a word to its integer value using the letter->digit map."""
```

```
    num = 0
```

```
    for ch in word:
```

```
        num = num * 10 + mapping[ch]
```

```
    return num
```

```
def solve_cryptarithm(addend1, addend2, result):
```

```
    """Brute-force search for a digit assignment that makes the equation true."""
```

```
    # collect all distinct letters
```

```
    letters = set(addend1 + addend2 + result)
```

```
    if len(letters) > 10:      # more letters than digits → impossible
```

```
        print("Too many different letters.")
```

```
        return None
```

```
    # first letters cannot be zero
```

```
    first_letters = {addend1[0], addend2[0], result[0]}
```

```
    digits = range(10)      # 0-9
```

```
    for perm in itertools.permutations(digits, len(letters)):
```

```
# create mapping letter -> digit
```

```
mapping = dict(zip(letters, perm))
```

```
# skip if any leading letter maps to 0
```

```
if any(mapping[ch] == 0 for ch in first_letters):
```

```
    continue
```

```
a = word_to_number(addend1, mapping)
```

```
b = word_to_number(addend2, mapping)
```

```
r = word_to_number(result, mapping)
```

```
if a + b == r:
```

```
    return mapping, a, b, r
```

```
return None # no solution found
```

```
if __name__ == "__main__":
```

```
    # Example problem: SEND + MORE = MONEY
```

```
    add1 = "SEND"
```

```
    add2 = "MORE"
```

```
    res = "MONEY"
```

```
    print("Trying to solve:", add1, "+", add2, "=", res)
```

```
    answer = solve_cryptarithm(add1, add2, res)
```

```

if answer is None:
    print("No solution found.")
else:
    mapping, a, b, r = answer
    print("\nSolution found!")
    print("Letter → digit mapping:")
    for ch in sorted(mapping):
        print(f" {ch} = {mapping[ch]}")
    print()
    print(f" {add1} = {a}")
    print(f" {add2} = {b}")
    print(f" {res} = {r}")
    print(f"Check: {a} + {b} = {r}  ->  {a + b == r}")

```

sample input

Trying to solve: SEND + MORE = MONEY

Ouput

Letter → digit mapping:

D = 7

E = 5

M = 1

N = 6

O = 0

R = 8

$$S = 9$$

$$Y = 2$$

$$\text{SEND} = 9567$$

$$\text{MORE} = 1085$$

$$\text{MONEY} = 10652$$

$$\text{Check: } 9567 + 1085 = 10652 \rightarrow \text{True}$$

EXPERIMENT 5: Write the python program for Missionaries Cannibal problem

Program

```
from collections import deque
```

```
def valid(m, c):
```

```
    if m < 0 or c < 0 or m > 3 or c > 3:
```

```
        return False
```

```
    if m > 0 and m < c:
```

```
        return False
```

```
    if (3-m) > 0 and (3-m) < (3-c):
```

```
        return False
```

```
    return True
```

```
def neighbors(state):
```

```
    ml, cl, boat = state
```

```
    moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]
```

```
    res = []
```

```
    for dm, dc in moves:
```

```
        if boat == 0:
```

```
            ns = (ml - dm, cl - dc, 1)
```

```
        else:
```

```
            ns = (ml + dm, cl + dc, 0)
```

```
        if valid(ns[0], ns[1]):
```

```
            res.append(ns)
```

```
return res
```

```
def solve():
```

```
    start = (3, 3, 0)
```

```
    goal = (0, 0, 1)
```

```
    q = deque()
```

```
    q.append((start, []))
```

```
    seen = {start}
```

```
    while q:
```

```
        state, path = q.popleft()
```

```
        if state == goal:
```

```
            return path + [state]
```

```
        for ns in neighbors(state):
```

```
            if ns not in seen:
```

```
                seen.add(ns)
```

```
                q.append((ns, path + [state]))
```

```
    return None
```

```
if __name__ == "__main__":
```

```
    sol = solve()
```

```
    if sol:
```

```
        print("Steps:", len(sol)-1)
```

```
        for i, (ml, cl, b) in enumerate(sol):
```

```
            side = "left" if b == 0 else "right"
```

```
            print(f'Step {i}: {ml}M {cl}C left, boat {side}')
```

```
    else:
```

```
print("No solution")
```

sample input:

Steps: 11

Output:

Step 0: 3M 3C left, boat left

Step 1: 3M 1C left, boat right

Step 2: 3M 2C left, boat left

Step 3: 3M 0C left, boat right

Step 4: 3M 1C left, boat left

Step 5: 1M 1C left, boat right

Step 6: 2M 2C left, boat left

Step 7: 0M 2C left, boat right

Step 8: 0M 3C left, boat left

Step 9: 0M 1C left, boat right

Step 10: 1M 1C left, boat left

Step 11: 0M 0C left, boat right

EXPERIMENT 6: Write the python program for Vacuum Cleaner problem

Program:

```
def perceive(location, status):  
    """  
    Returns the current percept: (location, status[location])  
    """  
    return (location, status[location])  
  
def rules_match(state):  
    """  
    Simple rule-based agent for the 2-location vacuum world.  
    Returns an action based on the current percept.  
    """  
    loc, clean = state  
    if clean == 'Dirty':  
        return 'Suck'  
    elif loc == 'A':  
        return 'Right'  
    elif loc == 'B':  
        return 'Left'  
    else:  
        return 'NoOp'
```

```
def update_location(loc, action):  
    """Update the vacuum's location based on the action."""  
    if action == 'Right':  
        return 'B'  
    elif action == 'Left':  
        return 'A'  
    else:  
        return loc
```

```
def update_status(loc, action, status):  
    """Update the cleanliness status after an action."""  
    if action == 'Suck':  
        status = status.copy()  
        status[loc] = 'Clean'  
    return status
```

```
def run_vacuum(initial_loc, initial_status, steps=10):  
    """  
    Simulate the vacuum agent for a given number of steps.  
    Prints each step so you can see what the agent does.  
    """  
    loc = initial_loc  
    status = dict(initial_status)
```

```

print(f"Initial state: Vacuum at {loc}, Status = {status}")

for step in range(1, steps + 1):
    percept = perceive(loc, status)
    action = rules_match(percept)
    print(f"Step {step:2d}: Percept {percept} -> Action {action}")

    if action == 'Suck':
        status = update_status(loc, action, status)
    else:
        loc = update_location(loc, action)

    print(f"        -> New state: Vacuum at {loc}, Status = {status}\n")

    if all(v == 'Clean' for v in status.values()):
        print("All squares are clean. Stopping.")
        break

if __name__ == "__main__":
    start_location = 'A'
    start_status = {'A': 'Dirty', 'B': 'Dirty'}

    run_vacuum(start_location, start_status, steps=15)

```

sample input:

Initial state: Vacuum at A, Status = {'A': 'Dirty', 'B': 'Dirty'}

Output:

Step 1: Percept ('A', 'Dirty') -> Action Suck

-> New state: Vacuum at A, Status = {'A': 'Clean', 'B': 'Dirty'}

Step 2: Percept ('A', 'Clean') -> Action Right

-> New state: Vacuum at B, Status = {'A': 'Clean', 'B': 'Dirty'}

Step 3: Percept ('B', 'Dirty') -> Action Suck

-> New state: Vacuum at B, Status = {'A': 'Clean', 'B': 'Clean'}

All squares are clean. Stopping.

EXPERIMENT 7: Write the python program to implement BFS.

Program:

```
from collections import deque

def bfs(graph, start):
    """Perform breadth-first search on a graph.
       graph: dict where key is a node and value is a list of neighbours.
       start: node to begin the search from.
    """
    visited = set()      # keep track of visited nodes
    queue = deque()      # queue for BFS
    order = []           # order in which nodes are visited

    visited.add(start)
    queue.append(start)

    while queue:
        node = queue.popleft()
        order.append(node) # record the visit

        for neighbour in graph.get(node, []):
            if neighbour not in visited:
                visited.add(neighbour)
```



```
queue.append(neighbour)
```

```
return order
```

```
if __name__ == "__main__":  
    # Example graph (undirected)  
    example_graph = {  
        'A': ['B', 'C'],  
        'B': ['A', 'D', 'E'],  
        'C': ['A', 'F'],  
        'D': ['B'],  
        'E': ['B', 'F'],  
        'F': ['C', 'E']  
    }
```

```
start_node = 'A'
```

```
visit_order = bfs(example_graph, start_node)
```

```
print("BFS traversal starting from", start_node, ":")
```

```
print(" -> ".join(visit_order))
```

Output:

BFS traversal starting from A :

A -> B -> C -> D -> E -> F

EXPERIMENT 8: Write the python program to implement DFS.

Program:

```
def dfs_iterative(graph, start):  
    if start not in graph:  
        return []  
  
    visited = set()  
    stack = [start]  
    visited.add(start)  
    result = []  
  
    while stack:  
        node = stack.pop()  
        result.append(node)  
        for neighbor in reversed(graph.get(node, [])):  
            if neighbor not in visited:  
                visited.add(neighbor)  
                stack.append(neighbor)  
  
    return result
```

```
# Example usage:

if __name__ == "__main__":
    # Example graph (undirected)
    graph = {
        'A': ['B', 'C'],
        'B': ['A', 'D', 'E'],
        'C': ['A', 'F'],
        'D': ['B'],
        'E': ['B', 'F'],
        'F': ['C', 'E']
    }

    print("DFS traversal starting from 'A':")
    traversal = dfs_iterative(graph, 'A')
    print(" -> ".join(traversal))
```

output:

DFS traversal starting from 'A':

A -> B -> D -> E -> F -> C

EXPERIMENT 9 Write the python to implement Travelling Salesman Problem

Program:

```
import sys
```

```
import math
```

```
from typing import List, Tuple
```

```
def tsp_held_karp(dist: List[List[float]]) -> Tuple[float, List[int]]:
```

```
    """
```

```
        Solve TSP exactly using the Held-Karp dynamic programming algorithm.
```

```
        Time   :  $O(n^2 * 2^n)$ 
```

```
        Space  :  $O(n * 2^n)$ 
```

```
        Parameters
```

```
        -----
```

```
        dist : square distance matrix (dist[i][j] = cost from i to j)
```

```
        Returns
```

```
        -----
```

```
        (min_cost, tour) where tour starts and ends at city 0.
```

```
    """
```

```
    n = len(dist)
```

```
    if n == 0:
```

```
        return 0.0, []
```

```
    if n == 1:
```

```
return 0.0, [0]
```

```
INF = float('inf')
```

```
dp = [[INF] * n for _ in range(1 << n)]
```

```
parent = [[-1] * n for _ in range(1 << n)]
```

```
dp[1][0] = 0.0
```

```
for mask in range(1 << n):
```

```
    if not (mask & 1):
```

```
        continue
```

```
    for u in range(n):
```

```
        if not (mask & (1 << u)):
```

```
            continue
```

```
            cur = dp[mask][u]
```

```
            if cur == INF:
```

```
                continue
```

```
            for v in range(n):
```

```
                if mask & (1 << v):
```

```
                    continue
```

```
                    nxt = mask | (1 << v)
```

```
                    new_cost = cur + dist[u][v]
```

```
                    if new_cost < dp[nxt][v]:
```

```
                        dp[nxt][v] = new_cost
```

```
                        parent[nxt][v] = u
```

```

full_mask = (1 << n) - 1
best_cost = INF
last = -1
for u in range(1, n):
    cost = dp[full_mask][u] + dist[u][0]
    if cost < best_cost:
        best_cost = cost
        last = u

if last == -1:
    return INF, []

# reconstruct path
path = [0]
mask = full_mask
cur = last
while cur != -1:
    path.append(cur)
    prev = parent[mask][cur]
    mask ^= (1 << cur)
    cur = prev
path.reverse()
path.append(0)
return best_cost, path

```

```
def tsp_nearest_neighbor(dist: List[List[float]], start: int = 0) -> Tuple[float, List[int]]:
```

```
    n = len(dist)
```

```
    if n == 0:
```

```
        return 0.0, []
```

```
    visited = [False] * n
```

```
    tour = [start]
```

```
    visited[start] = True
```

```
    cur = start
```

```
    total = 0.0
```

```
    for _ in range(n - 1):
```

```
        nxt = -1
```

```
        best = float('inf')
```

```
        for v in range(n):
```

```
            if not visited[v] and dist[cur][v] < best:
```

```
                best = dist[cur][v]
```

```
                nxt = v
```

```
        if nxt == -1:
```

```
            break
```

```
        visited[nxt] = True
```

```
        tour.append(nxt)
```

```
        total += best
```

```
        cur = nxt
```

```
    total += dist[cur][start]
```

```

    tour.append(start)

    return total, tour

if __name__ == "__main__":
    example_dist = [
        [0, 10, 15, 20],
        [10, 0, 35, 25],
        [15, 35, 0, 30],
        [20, 25, 30, 0]
    ]

    print("=== Held-Karp (exact) ===")
    cost, path = tsp_held_karp(example_dist)
    print(f"Minimum cost: {cost}")
    print(f"Tour: {path}")

    print("\n=== Nearest Neighbor (heuristic) ===")
    cost2, path2 = tsp_nearest_neighbor(example_dist)
    print(f"Tour cost: {cost2}")
    print(f"Tour: {path2}")

```

output:

Minimum cost: 80.0

Tour: [0, 2, 3, 1, 0, 0]

=== Nearest Neighbor (heuristic) ===

Tour cost: 80.0

Tour: [0, 1, 3, 2, 0]

EXPERIMENT 10: Write the python program to implement A* algorithm

Program:

```
import heapq

def a_star(graph, start, goal, heuristic):
    open_list = []
    heapq.heappush(open_list, (0, start))
    came_from = {}
    g_cost = {node: float('inf') for node in graph}
    g_cost[start] = 0
    f_cost = {node: float('inf') for node in graph}
    f_cost[start] = heuristic[start]

    while open_list:
        current = heapq.heappop(open_list)[1]
        if current == goal:
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            path.reverse()
            return path, g_cost[goal]

        for neighbor, cost in graph[current]:
```

```
tentative_g = g_cost[current] + cost
```

```
if tentative_g < g_cost[neighbor]:
```

```
    came_from[neighbor] = current
```

```
    g_cost[neighbor] = tentative_g
```

```
    f_cost[neighbor] = tentative_g + heuristic[neighbor]
```

```
    heapq.heappush(open_list, (f_cost[neighbor], neighbor))
```

```
return None, float('inf')
```

```
graph = {
```

```
    'A': [('B', 1), ('C', 3)],
```

```
    'B': [('A', 1), ('D', 3), ('E', 6)],
```

```
    'C': [('A', 3), ('F', 5)],
```

```
    'D': [('B', 3)],
```

```
    'E': [('B', 6), ('G', 2)],
```

```
    'F': [('C', 5), ('G', 2)],
```

```
    'G': [('E', 2), ('F', 2)]
```

```
}
```

```
heuristic = {
```

```
    'A': 7,
```

```
    'B': 6,
```

```
    'C': 4,
```

```
    'D': 5,
```

```
'E': 2,  
'F': 1,  
'G': 0  
}  
start = 'A'  
goal = 'G'  
  
path, cost = a_star(graph, start, goal, heuristic)  
  
if path:  
    print("Shortest Path :", " -> ".join(path))  
    print("Total Cost   :", cost)  
else:  
    print("No path found.")
```

output:

Shortest Path : A -> B -> E -> G

Total Cost : 9

EXPERIMENT 11: Write the python program for Map Coloring to implement CSP.

Program:

```
def is_safe(node, color, assignment, graph):  
    for neighbor in graph[node]:  
        if neighbor in assignment and assignment[neighbor] == color:  
            return False  
    return True  
  
def map_coloring(graph, colors, assignment, nodes, index):  
    if index == len(nodes):  
        return True  
  
    node = nodes[index]  
  
    for color in colors:  
        if is_safe(node, color, assignment, graph):  
            assignment[node] = color  
  
            if map_coloring(graph, colors, assignment, nodes, index + 1):  
                return True  
            del assignment[node]  
  
    return False  
  
graph = {  
    'A': ['B', 'C', 'D'],
```

```

    'B': ['A', 'C', 'E'],
    'C': ['A', 'B', 'D', 'E'],
    'D': ['A', 'C', 'E'],
    'E': ['B', 'C', 'D']
}

colors = ['Red', 'Green', 'Blue']

assignment = {}
nodes = list(graph.keys())

if map_coloring(graph, colors, assignment, nodes, 0):
    print("Map Coloring Solution:\n")
    for region in nodes:
        print(f"{region} --> {assignment[region]}")
else:
    print("No solution exists.")

```

output:

Map Coloring Solution:

A --> Red

B --> Green

C --> Blue

D --> Green

E --> Red

EXPERIMENT 12: Write the python program for Tic Tac Toe game

Program

Tic Tac Toe using Minimax Algorithm

```
import math
```

Display the board

```
def print_board(board):
```

```
    for i in range(3):
```

```
        print(" | ".join(board[i]))
```

```
        if i < 2:
```

```
            print("-" * 9)
```

Check winner

```
def check_winner(board):
```

```
    # Rows
```

```
    for row in board:
```

```
        if row[0] == row[1] == row[2] != '':
```

```
            return row[0]
```

```
    # Columns
```

```
    for col in range(3):
```

```
        if board[0][col] == board[1][col] == board[2][col] != '':
```

```
            return board[0][col]
```

```
# Diagonals
```

```
if board[0][0] == board[1][1] == board[2][2] != '':
```

```
    return board[0][0]
```

```
if board[0][2] == board[1][1] == board[2][0] != '':
```

```
    return board[0][2]
```

```
return None
```

```
# Check if board is full
```

```
def is_full(board):
```

```
    for row in board:
```

```
        if ' ' in row:
```

```
            return False
```

```
    return True
```

```
# Minimax Algorithm
```

```
def minimax(board, is_max):
```

```
    winner = check_winner(board)
```

```
    if winner == 'O':
```

```
        return 1
```

```
    elif winner == 'X':
```

```
        return -1
```

```
    elif is_full(board):
```



```
return 0
```

```
if is_max:
```

```
    best = -math.inf
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            if board[i][j] == '':
```

```
                board[i][j] = 'O'
```

```
                best = max(best, minimax(board, False))
```

```
                board[i][j] = ''
```

```
    return best
```

```
else:
```

```
    best = math.inf
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            if board[i][j] == '':
```

```
                board[i][j] = 'X'
```

```
                best = min(best, minimax(board, True))
```

```
                board[i][j] = ''
```

```
    return best
```

```
# Find the best move
```

```
def best_move(board):
```

```
    best_score = -math.inf
```

```
    move = (-1, -1)
```

```

for i in range(3):
    for j in range(3):
        if board[i][j] == ' ':
            board[i][j] = 'O'
            score = minimax(board, False)
            board[i][j] = ' '

            if score > best_score:
                best_score = score
                move = (i, j)

    return move

board = [[' ' for _ in range(3)] for _ in range(3)]

print("Tic Tac Toe")
print("You are X")
print("Computer is O\n")

while True:
    print_board(board)

    # Player Move
    while True:
        row = int(input("\nEnter row (0-2): "))
        col = int(input("Enter column (0-2): "))

```

```

if 0 <= row <= 2 and 0 <= col <= 2 and board[row][col] == ' ':
    board[row][col] = 'X'
    break
else:
    print("Invalid move. Try again.")

if check_winner(board) == 'X':
    print_board(board)
    print("\nYou Win!")
    break

if is_full(board):
    print_board(board)
    print("\nMatch Draw!")
    break

# Computer Move
row, col = best_move(board)
board[row][col] = 'O'

print("\nComputer played...\n")

if check_winner(board) == 'O':
    print_board(board)
    print("\nComputer Wins!")
    break

```

```
if is_full(board):  
    print_board(board)  
    print("\nMatch Draw!")  
    break
```

input

Tic Tac Toe

You are X

Computer is O

| |

| |

| |

Enter row (0-2): 1

Enter column (0-2): 1

Output:

Computer played...

O | |

| X |

| |